MyCobot280Pi Control System

ROS 2 manipulator control | Six degrees of freedom | Physical robot and virtual-model interfaces

Project status: Modular development implementation with code, launch instructions, bench photographs, and GUI/RViz screenshots. Hardware migration history follows the author's résumé; the uploaded source snapshot documents several control variants.

1) Project overview

The project develops flexible joint-space and Cartesian command paths for a myCobot 280 Pi. It separates command generation from robot communication and model visualization so that a GUI or script can supply targets to a selected physical or virtual backend. An asynchronous MoveIt 2 inverse-kinematics client converts Cartesian pose requests into ordered joint-space commands.

The hardware work progressed from onboard Raspberry Pi control to an external PC and later NVIDIA Jetson Orin Nano. The software work includes dedicated ROS 2 interface nodes, launch composition, unit conversions, serial access, joint-state publication, and integrating a pre-existing virtual-controller GUI.



Figure 1. Bench setup showing the robot, external computer, and visualization. Author-supplied photograph.

The author's contribution is the custom interfaces, integration, and control-platform development. The GUI, robot model/vendor ROS packages, pymycobot API, and MoveIt provide reused functionality. Model visualization is distinguished from physical feedback and from physics-based simulation.

2) Hardware evolution and communication

Development phase	Implementation described by the author
Original onboard platform	myCobot 280 Pi with Raspberry Pi 4B.
Onboard upgrade	Replaced Raspberry Pi 4B with Raspberry Pi 5.
External PC	Moved primary control to a PC; CH340 USB-to-TTL connected the host to the robot auxiliary ESP32 over UART.
External Jetson	Migrated to NVIDIA Jetson Orin Nano; used an Arduino Uno UART bridge when the CH340 converter was not recognized.

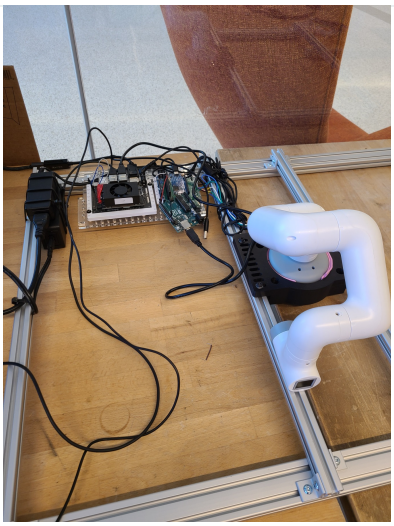


Figure 2. Cobot and external control hardware arranged on the bench.

The custom physical controller uses MyCobot280 from pymycobot with a default baud rate of 1,000,000. Port selection varies between workflows: parameterized USB paths appear in the hardware interface, while a dedicated Cartesian variant opens /dev/ttyAMA0. A launch file declaring a port argument does not by itself mean every node consumes that argument.

The Arduino bridge history is supported by the résumé and author account. Bridge firmware is not included in the submitted source snapshot, so its buffering, electrical implementation, and achieved throughput cannot be independently described from these files. Restoring communication is not presented as a measured latency reduction.

3) ROS 2 software architecture

Component/file	Main responsibility
hard_control.py	GUI-oriented physical controller with joint/Cartesian enable topics and serial API access.
soft_control.py	GUI-oriented virtual controller with optional Cartesian IK and joint-state publication.
robot_joint_interface.py	Physical joint command subscriber and feedback publisher.
robot_joint_interface_vir.py	Virtual joint command subscriber; publishes target states without querying hardware.
robot_cart_interface.py	Dedicated physical Cartesian-offset interface.
ik_solver.py	Standalone /cart_cmd -> MoveIt IK -> /joint_cmd conversion.
des_jstate_publisher.py	User-editable joint targets from a Python script.
des_cstate_publisher.py	User-editable Cartesian position offsets from a Python script.
rover_launch	Composes robot description, backend nodes, MoveIt, and RViz for different workflows.

Topics and contracts

Topic	Meaning in this project
/joint_cmd	Float64MultiArray containing six joint angles in radians.
/cart_cmd	Twist fields repurposed for position and Euler-angle offsets, not velocity commands.
/joint_en, /cart_en	Bool mode flags used by GUI-oriented controllers.
/joint_states	JointState for hardware feedback or virtual targets, depending on active backend.
/compute_ik	MoveIt GetPositionIK service used for pose-to-joint conversion.

Physical and virtual backends are separate execution choices. Running competing publishers on /joint_states without deliberate coordination can mix state sources. The supplied screenshots illustrate workflows; they do not establish an automatically synchronized digital twin.

4) Interactive GUI integration

A pre-existing virtual-controller GUI provides joint and Cartesian slider modes. The author developed ROS 2 interface and launch integration connecting that GUI to the robot-control system. Joint controls provide angle inputs; Cartesian controls provide X/Y/Z and roll/pitch/yaw offsets relative to configured reference poses.

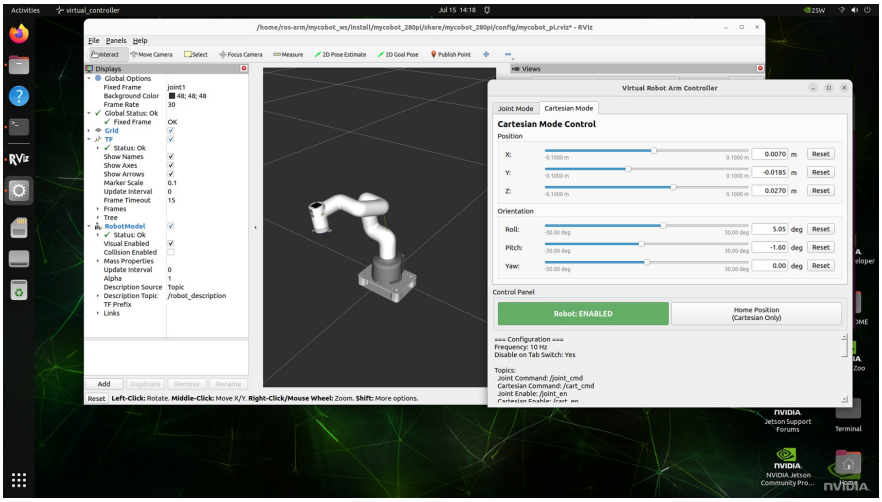


Figure 3. Cartesian slider mode beside the RViz model. Source: Dak1.png.

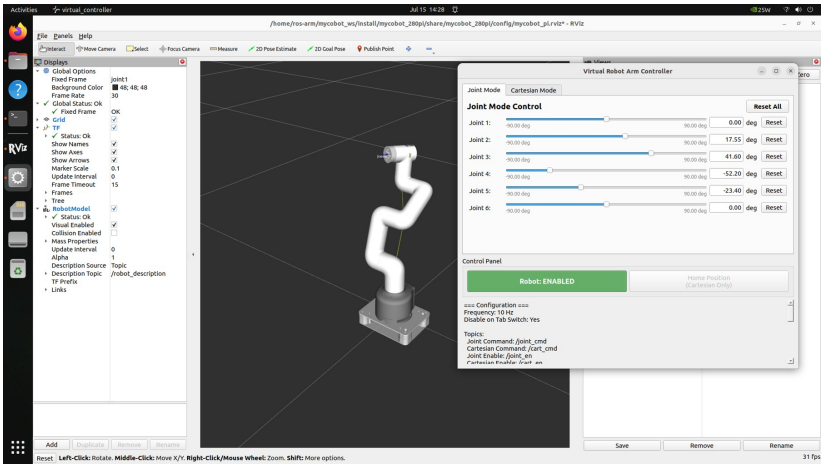


Figure 4. Joint-angle slider mode with a different model pose. Source: Dak3.png.

The GUI-oriented controllers subscribe to both enable flags and command topics. hard_control accepts a joint command only when joint mode is enabled and Cartesian mode is disabled, and applies the converse condition to Cartesian commands. soft_control switches between its reference joint state and reference Cartesian state when modes are enabled.

The GUI process is commented out in several rover_launch variants and can be started separately with ros2 launch teleop_core_virtual virtual_controller.launch.py. Documentation must therefore distinguish launching the backend from launching the command interface. GUI authorship is not attributed to this project.

5) Physical robot command and feedback paths

Joint-space control

Incoming /joint_cmd values are checked for a six-value command, converted from radians to degrees, and passed to the robot API using send_angles. The code uses an API speed argument of 50; this value is not treated as a measured physical velocity. Serial API operations are wrapped with a thread lock to serialize access.

Feedback publication

safe_get_angles retries up to three times and accepts only list/tuple responses with six elements. Valid joint angles are converted back to radians and published under named robot joints in /joint_states. A 0.1-second timer requests feedback publication; blocking API calls and sleeps can affect its delivered rate.

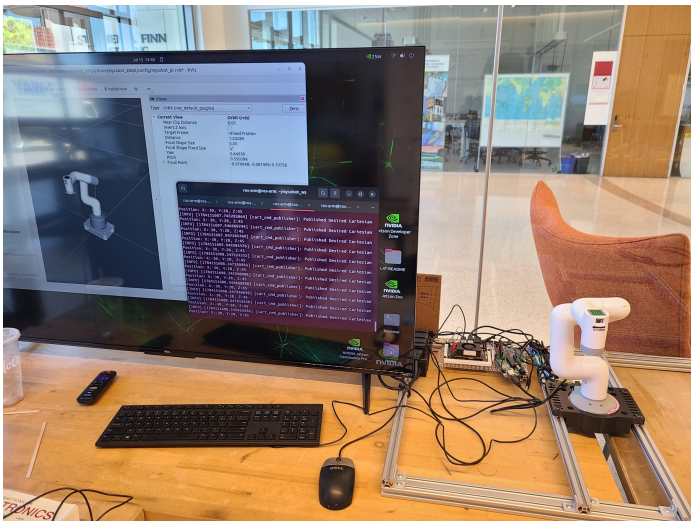


Figure 5. Physical robot, model visualization, and active terminal output in the bench setup.

Direct Cartesian variant

The dedicated Cartesian path initializes the robot with a joint command, reads a reference Cartesian pose, then adds incoming offsets. Linear fields are converted from meters to millimeters; angular fields from radians to degrees. Orientation targets are wrapped to the [-180,180) interval and sent through send_coords. Dispatch is limited by a 0.2-second interval.

Joint-state feedback on this path comes from the hardware API, whereas virtual-model nodes publish commanded joint values. Neither a target message nor a displayed model pose alone proves that the physical end effector reached the requested pose within a quantified error.

6) MoveIt 2 inverse kinematics

ik_solver.py subscribes to /cart_cmd, adds the received offsets to a configured home pose [0.1692, -0.06462, 0.1964, pi, 0, -pi/2], and constructs a PoseStamped request in frame g_base. Euler angles are converted to quaternion orientation through quaternion_from_euler.

IK request element	Configured behavior
Service and type	/compute_ik; moveit_msgs/GetPositionIK.
Planning group	arm_group.
Seed state	Previous target joint values in the expected six-joint order.
Timeout field	1 second.
Collision option	avoid_collisions=False in the submitted code.
Dispatch gating	0.1-second interval, change tolerance, and an in-flight flag.

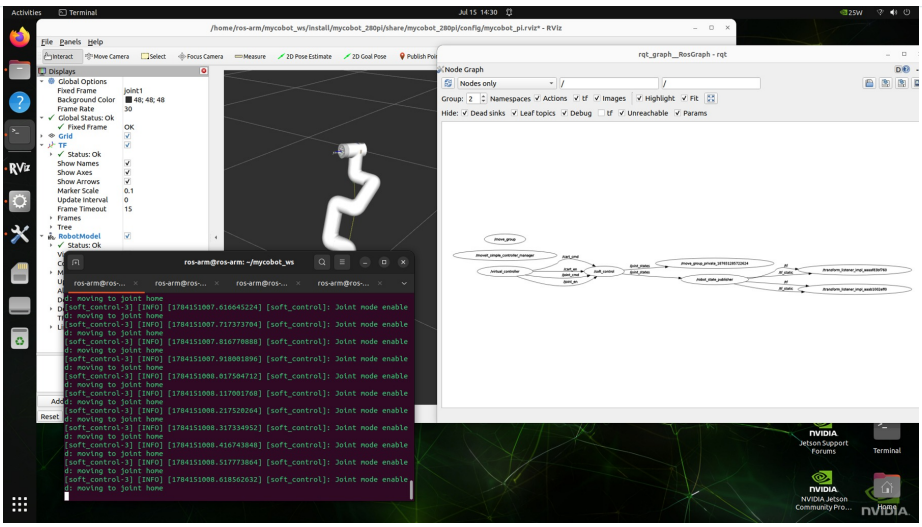


Figure 6. RViz, ROS graph, and terminal activity during development. Source: Dak4.png.

The asynchronous callback maps response joint names to values, reorders them to match the control interface, publishes /joint_cmd, and updates the remembered target pose. The in-flight flag prevents concurrent requests through that node. The standalone node can discard new callbacks while a solve is busy; soft_control additionally checks the latest target after completion.

This is target-level inverse kinematics, not a complete time-parameterized trajectory execution pipeline. The implementation checks for an empty result but should also inspect MoveIt error codes and validate limits/reachability. Collision-aware physical execution is not established when collision avoidance is disabled in the request.

7) Script interfaces and launch procedure

The script publishers decouple target specification from the selected robot backend. In des_jstate_publisher.py the example command [0,60,-90,30,0,0] is expressed in degrees, converted to radians, and published every 0.1 seconds. The one-shot guard is commented out, so the provided example repeatedly publishes.

des_cstate_publisher.py uses example offsets x=-30, y=20, z=45 mm, converts them to meters, and publishes /cart_cmd. Its orientation offsets are zero in the submitted example. The receiver interprets these values relative to its configured home pose rather than as an absolute global position.

Workflow	Command from supplied instructions
Build/source	colcon build --symlink-install; source the workspace environment.
Physical joint backend	ros2 launch rover_launch mycobot_280pi.launch.py
Virtual joint backend	ros2 launch rover_launch mycobot_280pi_vir.launch.py
Joint input	ros2 run mycobot_interface goal_joint_state_pub
IK conversion	ros2 run mycobot_interface ik_solver
Cartesian input	ros2 run mycobot_interface goal_cart_state_pub

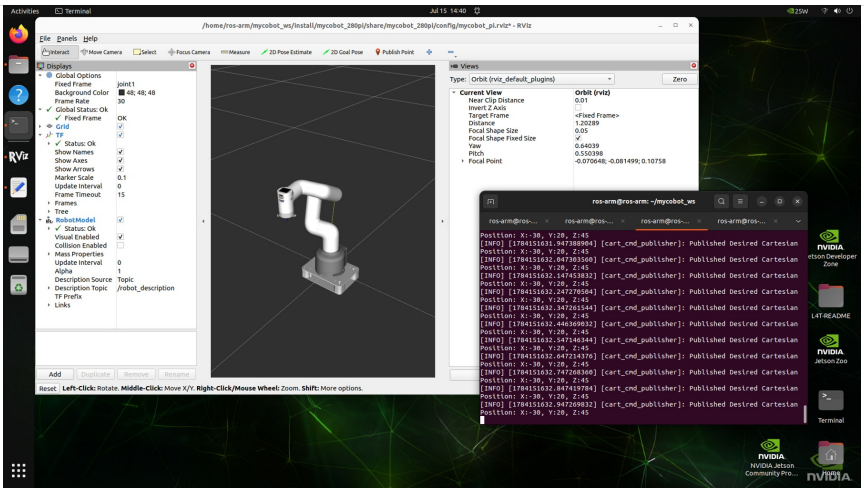


Figure 7. Command-driven robot visualization and terminal messages. Source: Dak7.png.

For Cartesian script input, start the selected backend and its MoveIt service before the IK node and command publisher. For GUI workflows, select the corresponding hardware or virtual GUI-oriented launch variant and launch the controller GUI separately where required. Port, baud, URDF, and RViz settings must match the selected host.

8) Results, limitations, and future work

Documented outcome

The code, instructions, five hardware photographs, and eight screenshots document an integrated development system with multiple command paths. The author reports smooth, responsive control and real-time visualization; the archive provides no controlled end-to-end latency, trajectory tracking, or physical repeatability benchmark, so numerical performance claims are not added.

Current engineering limits

Blocking sleeps and serial calls can delay callbacks. Mode state and command validity need systematic testing. Some launch variants declare arguments not consumed by all nodes, and several optional nodes are commented out. MoveIt is configured with simulation-time arguments in parts of the launch stack; clock availability must be checked in a live deployment. Reused helper functions in virtual files are not necessarily active hardware logic.

Recommended future work

Measure command-to-motion and feedback latency on each host/bridge; validate joint and Cartesian limits; use explicit command-completion feedback; handle IK error codes and collision checking; reduce callback blocking; and document timestamp/frame conventions. Add tests for malformed commands, serial disconnects, unreachable poses, and repeated mode switching.

Source register

MyCobot280Pi Control System.zip: nested src-ros2_ws-may4.zip, especially src/mycobot_interface and src/rover_launch; RvizCommands.txt; Images/20260804_*.jpg; Screenshots/Dak1.png-Dak8.png. Reviewed robot-interface nodes, hardware/virtual controllers, publishers, IK client, and launch definitions. Vendor descriptions, robot models, MoveIt components, and teleop_core_virtual are reused dependencies. Résumé: Dak Amarasinghe Resume(20261030-094031).pdf.

Conclusion

The project demonstrates modular manipulator-control integration across software and hardware platforms. Its strongest technical elements are explicit topic interfaces, physical/virtual backend separation, unit conversion, asynchronous IK integration, and practical serial communication troubleshooting.

Preparation method: source and image inspection; no ROS runtime, physical robot, or bridge firmware execution was performed.